

Six Birds Foundations VII: Lawful Theory Interaction

Ioannis Tsiokos 

6 October 2026 · Preprint v2.0

Archived draft DOI: doi:10.5281/zenodo.23187439

Abstract

When may two theories interact lawfully, and what must a joint theory show to count as more than its parts? This paper gives a typed certificate language for such claims and proves where mathematics backs its certificates. The language keeps apart claims that are easily conflated: that content is expressible, present, reachable or occurrent; that two theories touch or form a joint theory; that one enables another or determines it; that order matters or time has an arrow. Twenty-seven named countermodels in a finite reference world exhibit these separations. An observable of a join is strict, factoring on its image through neither parent nor their pairing, exactly when it has a split pair against each of the three maps. When admitting an item can only enable further admissions, every admission order reaches the same terminal state, by Newman’s lemma, and one disabling guard produces two. As a soundness check, a semantic strict join satisfies the language’s strictness certificate once it is also proved not to factor, on its image, through a declared refinement and schedule, which strictness alone does not give. Other entries are definitions or recorded refusals. The results are formalized in Lean 4, with the exceptions stated in Section 10.

Keywords. emergence calculus; lawful theory interaction; join of theories; strict join; factorization; split pair; confluence; Newman’s lemma; certificate language; enablement; access coordinates; Six Birds Theory.

1 Introduction

Two theories are given, each internally lawful. When may they interact, and when does their interaction produce a new theory rather than a restatement of the two old ones? The question arises wherever theories are combined: two axiomatizations merged into one, two models of a system coupled through a shared instrument, two layers of description linked by an interface. It is easy to answer carelessly. Two theories that share an origin, a carrier or a vocabulary are said to interact; a combination that produces some new quantity is said to be a genuine joint theory; a combination that removes an old difficulty is said to have fewer problems. Each inference takes something real and observable and reads more into it than follows.

This paper does two things about that situation. First, it gives a *certificate language*: typed records in which a claim about interaction must be written down, with a separate field for each thing that is easily conflated. A record says, for instance, whether content is expressible, present, reachable or occurrent; whether two theories are merely in contact or have formed a composite; whether an enabling theory also determines what it enables; whether a dependence on order has been shown or an arrow of time. A finite reference world exhibits, for each separation, a configuration that has one property and lacks the other. Second, it proves where such certificates have mathematical content, by giving semantic models in which the key certificates are backed by theorems.

The catalog. The certificates are organized in a *catalog*: a list of 36 entries, each a claim about lawful interaction together with the certificate that must back it, the finite evidence attached to it and its counterpart in the Lean development. The entries fall into four clusters (access and admission, contact and join, enablement and descent, dynamics and residue) and are anchored by one finite reference world (Section 8). It is worth being plain about what the catalog contains. Most entries fix what a certificate must record and when it counts as valid; they are definitions, and the statements drawn from them are short consequences of those definitions or refusals of credit that a definition withholds. Table 2 classifies every result by what it is. The mathematics lies in the semantic results of Sections 4 and 5, which the introduction leads with.

1.1 A worked example

Two theories and a join. Let theory A observe one bit a and theory B observe one bit b . A candidate joint theory has states $j = (a, b, h)$ with a third bit h , and maps $p_A(j) = a$ and $p_B(j) = b$ back to the two parents. Which observables of the joint theory are genuinely new?

The observable $f(j) = a$ is not new: it factors through p_A on its image. The observable $f(j) = h$ is. The two states $(0, 0, 0)$ and $(0, 0, 1)$ have the same image under p_A , under p_B , and under the pair (p_A, p_B) , yet they differ in h . No function of a , of b , or of (a, b) can therefore return h . We call such a pair of states a *split pair*. Theorem 4.3 shows that split pairs are exactly what stands in the way of factorization on the reached image, so that h is *strict* in the sense of Definition 4.4 (Example 4.6).

The pairing matters. An observable such as $a \oplus b$ factors on its image through neither parent alone, since a does not determine $a \oplus b$ and neither does b , but it factors on its image through the pair (a, b) . It is information that the two theories already hold jointly, and a joint theory that only adds such observables adds nothing that the two parents do not jointly determine. This is why strictness is tested against the pairing as well as against each parent. What the test leaves open is stated in Remark 4.9.

Admitting the pieces. Suppose now that the joint theory is assembled by admitting items one at a time, where an item may be admitted once none of its prerequisites is still pending. Does the order of admission matter? If admitting an item can only enable further admissions, it does not: two admissions available at the same moment commute, every admission run terminates, and by Newman’s lemma every maximal order ends in the same state (Theorem 5.8). A single guard that lets one admission *disable* another breaks this. With items a and b , where b may be admitted only while a is still pending, admitting a first strands b , while admitting b first admits both. The two orders end in different terminal states (Example 5.9).

1.2 Main results

We state the headline results informally here; the precise statements carry all hypotheses.

Theorem (A: strictness is non-factorization). *An observable f of a join factors through a map q on the image of q exactly when no split pair for q and f exists (Theorem 4.3). Consequently f is strict, factoring on its image through neither parent nor their pairing, exactly when it has a split pair against each of the three maps (Theorem 4.5). The third bit of the three-bit join is strict and the first is not (Example 4.6). Strictness rules out relabelling and coarsening of a parent (Proposition 4.11); Remark 4.9 states its limit.*

Theorem (B: admission order does not matter when admission only enables). *In any system where an item may be admitted once none of its prerequisites is still pending, every run terminates,*

simultaneous admissions commute, and every maximal admission order from a given state reaches the same terminal state with the same audit value (Theorem 5.8). The proof applies Newman’s lemma (Theorem 5.2); local confluence of a finite system is decidable by a finite search (Proposition 5.4). Termination cannot be dropped (Example 5.6), and one disabling guard produces two terminal states (Example 5.9).

Theorem (C: a soundness check for the strictness certificate). *Let f be a strict observable of a semantic join, let q be a declared common refinement and s a declared schedule, and suppose it is proved that f factors on its image through neither q nor s . Then the anti-product witness whose flags are the truth values of the corresponding semantic predicates is valid (Theorem 4.12). Strictness does not supply the two extra hypotheses (Remark 4.9).*

Theorem (D: what the certificate language separates). *The certificate language keeps apart seven access coordinates, contact from join, composite formation from parent retention, enablement from descent (a recorded claim that an enabling theory also determines what it enables), reachability from occurrence, and order dependence from an arrow. Each separation is exhibited in the finite reference world by a named configuration with one property and not the other (Section 7, Table S2).*

Theorem A is the mathematics behind the catalog’s central object, the strict join. Theorem B is the mathematics behind its confluence entry. Theorem C is a soundness check between the two levels. Once its hypotheses are granted, each flag is settled in a line by the results behind Theorem A or by a hypothesis, so the theorem is close to a restatement. What it checks is that the Boolean certificate asks for nothing a semantic model cannot supply: its validity predicate is satisfied by a genuine model, with each flag the truth value of a proved statement.

1.3 Map of results

Table 1 lists the numbered results. Each is proved where it is stated.

Table 1. Map of results.

Result	What it establishes
Theorem 4.3	A map factors through another on its image exactly when no split pair exists; for a finite list with decidable equality, a scan decides it.
Theorem 4.5	An observable of a join is strict exactly when it has split pairs against each parent and against the pairing.
Proposition 4.8	An observable that factors globally through the pairing factors through every common refinement.
Proposition 4.11	A strict observable is neither relabelling-only nor coarsening-only.
Theorem 4.12	For a strict observable that does not factor through the declared refinement and is not scheduling-only, setting each flag of an anti-product witness to its semantic truth value, with a nonempty description and an audit entry, gives a valid witness; the three-bit join is an instance.
Theorem 5.2	A terminating, locally confluent relation is confluent (classical).
Corollary 5.3	Under confluence normal forms reachable from one state coincide, and with termination each state has exactly one.
Proposition 5.4	On a finite list with decidable equality and a decidable step relation, reachability and local confluence are decided by enumeration.
Theorem 5.8	In every admission system, admission runs terminate and every maximal admission order from a given state reaches the same terminal state.
Proposition 7.1	An upper step can be enabled by the lower dynamics while an upper observable does not factor through a lower quotient.
Proposition 7.2	A join that requires a capability not initially available, and alone produces it, never occurs in a lawful sequence.

Result	What it establishes
Proposition 7.3	Selection by intersection creates no fact.
Proposition 7.4	A bounded scan is exactly as strong as the coverage of its list and the closure of its horizon.
Proposition 7.5	With no admitted seed, no reachable generator and no open provision, no first extension is authorized.

1.4 Relation to earlier work

The factorization criterion of Theorem A is the classical fact that a map is constant on the fibres of another exactly when it factors through that map on its image; this form on the reached image recurs throughout the Six Birds series [25]. Newman’s lemma is classical [16], as is its standard counterexample without termination (Example 5.6) and the use of critical pairs to decide local confluence [12, 11]. The semantic package of Definition 4.1 is the theory package of Foundations I stripped to its observable [22]. New here are the semantic join and its strictness characterization (Theorem 4.5), which gives the catalog’s anti-product condition a meaning; the admission model with Theorem 5.8, the semantic content of the catalog’s confluence entry, and its disabling counterexample; the finite decision procedure in this setting; and the soundness check of Theorem 4.12.

The certificates sharpen records of the earlier papers. The access coordinates **exposed**, **recoverable** and **admissible** sharpen Foundations IV’s access quotient and Foundations II’s admissible claims, and the split into **expressible**, **present**, **reachable** and **occurrent** is new [25, 23]. Contact surfaces and join statuses sharpen the instrument-indexed records and status families of Foundations III [24]. The strict-join certificate sharpens strict extension as non-factorization [25, 24, 29]. The enablement record sharpens the distinction between enablement and within-layer causation in the agenthood paper [21]. The drive certificate sharpens the protocol trap of Foundations I and the no-go results on arrows from route residue [22, 28]. The bootstrap refusal sharpens the internalization law of Foundations V and no-free-distinction of Foundations IV [26, 25]. Section 2 places the certificate language among combination methods for logical theories, interface theories and provenance.

1.5 Roadmap

Section 2 discusses related work and Section 3 fixes the setting and classifies the results by status. Section 4 proves Theorems A and C, and Section 5 proves Theorem B. Section 6 presents the certificate language and Section 7 what it separates and refuses. Section 8 describes the finite reference world, Section 9 states the scope of the results, and Section 10 describes the Lean formalization. The companion supplement holds the source concordance, the full field lists of the certificates, the per-entry register, the no-go register, the finite evidence and the replay commands.

2 Related work

Combining and comparing theories. Several mature literatures address neighbouring, more specialized problems. Nelson–Oppen combination gives conditions under which decision procedures for quantifier-free fragments of suitably separated first-order theories cooperate [15]. Institution theory and algebraic specification study signatures, sentences, models, satisfaction and morphisms across logical systems, and build structured specifications from them [9, 20]. Interface automata and contract-based design make compatibility, refinement, environment assumptions and composition explicit [7, 1, 4]. Theories can also be compared without being composed: Morita and categorical equivalence give criteria for when two formulations are the same theory, and show how much

structure a criterion preserves [3, 30]. These are the nearest comparison classes. None of them is used here as a representation theorem for the join of Section 4, whose content is the elementary statement on factorization over the reached image in Theorem A.

Rewriting. Newman’s lemma [16] and the critical-pair method [12, 11] are the classical tools for proving that the order of rewriting steps does not matter. Section 5 applies them to admission orders.

Provenance, causation and enablement. Database provenance tracks how outputs depend on source data; causal models separate common causes from stronger causal claims; organizational-closure work treats constraints as acting on processes while depending on one another for their maintenance [10, 18, 14]. The certificate language keeps these apart: its source-independence field is a record of distinct ancestry, not statistical independence, and its enablement record is not a causal or biological closure model. The reachability and horizon fields are informed by the distinction between safety and liveness and by bounded model checking [2, 6, 5], in particular by the warning that a bounded search establishes a bounded result. Preregistration in scientific method motivates the prospective-commitment record [17].

The Six Birds series. Foundations I supplies the notion of a theory package: a carrier with a lens, a completion endomap and an audit [22]. Foundations II supplies the vocabulary of admissible claims, of which the admission records here are a specialization [23]. Foundations III supplies an audited interaction calculus within a fixed setting [24]; this paper asks the prior question of when interaction across a boundary is lawful at all. Foundations IV supplies the catalog form and the access quotient [25]. Foundations V uses the word *join* for the repair join, a lattice operation on quotients of one carrier [26]; the join of theories here is a different object and gets a different symbol, $\bowtie$ rather than $\vee$. Foundations V and VI provide formal libraries that the Lean development builds on [26, 27]. Individual certificates sharpen results of earlier papers in the series.

3 Setting

3.1 Theories, regimes and audits

A *theory package* in the sense of Foundations I is a carrier together with a lens that fixes which distinctions are kept, a completion endomap whose fixed points are the package’s objects, and an audit [22]. The semantic results of Sections 4 and 5 need much less: a carrier with an observable map, a join given by maps to the parents, and a step relation on states. The certificate language of Section 6 records claims about packages without modelling their internal structure.

An *admissibility regime* is the declared standard against which an interaction is judged: which operations are lawful, which sources may be drawn on, which costs must be paid and what the audit must record. Every existence, obstruction and cost claim below is relative to a declared regime, and none transfers to another regime without a record of what the change preserves.

An *audit* is an append-only list of entries, each with a subject, a disposition and a source location. Append-only matters: a failed, revoked or withdrawn entry remains in the record, so a later reader can see that something was tried and refused.

3.2 Certificates and theorems

The catalog’s objects are *certificates*: typed records with Boolean and data fields, usually with an audit. A certificate comes with a validity predicate, usually a conjunction saying that the right fields are set and, where the record has one, that the audit is non-empty. Many statements about certificates are therefore immediate: that a valid certificate has a given field set, that a credit whose definition requires a field is refused when the field is absent, or that a criterion is equivalent to the definition it restates. The supplement states what each of the catalog’s 36 entries establishes (Supplement Section S6).

Status by mathematics. Each result has one of three statuses, decided by the mathematics.

- A *theorem with an instance* is a general statement together with a concrete system, written in this paper, in which its hypotheses are proved, so that the conclusion is established for that system.
- A *general theorem* is a statement whose hypotheses are ordinary data that can be checked system by system; where it has classical instances, they rest on cited results.
- A *certificate specification* fixes what a record must contain and when it is valid, together with the consequences and refusals that follow from that definition. Like a template, it says nothing about a particular system until the record’s fields have been established for that system; establishing them is the *open obligation* of the specification.

Which results are also checked in Lean is a separate question, recorded in the last column of Table 2 and in Section 10. The instances in this paper are small finite models; no instance drawn from a particular scientific theory is carried out.

Table 2. The results by status.

Result	Status	Instance or open obligation	Checked in Lean
Split-pair criterion (Theorem 4.3)	general theorem	classical; the three-bit join (Example 4.6)	the criterion and the finite scan
Strictness by split pairs (Theorem 4.5)	theorem with an instance	the three-bit join (Example 4.6)	the general statement and the example; not the remark on $a \oplus b$
Refinement (Proposition 4.8, Remark 4.9)	general theorem	the identity refinement of the three-bit join shows the converse fails	both
Cheap constructions (Proposition 4.11)	general theorem	none needed	relabelling and coarsening
Soundness of the strictness certificate (Theorem 4.12)	theorem with an instance	the three-bit join with $q = \langle p_A, p_B \rangle$ and $s = p_A$	the general statement and the example
Newman’s lemma and unique normal forms (Theorem 5.2, Corollary 5.3)	general theorem	classical; termination is needed (Example 5.6)	both, and the four-state example
Finite decision (Proposition 5.4)	general theorem	none needed	reachability, joinability and peak checks
Admission order (Theorem 5.8)	theorem with an instance	admission without constraints (Example 5.10); one disabling guard lies outside the hypothesis (Example 5.9)	all four parts and both examples
Enablement without descent (Proposition 7.1)	theorem with an instance	a three-state model	both parts and the positive criterion
No free join (Proposition 7.2)	general theorem	a one-event example with the capability supplied	the exclusion and the example

Result	Status	Instance or open obligation	Checked in Lean
Selection (Proposition 7.3)	general theorem	insertion is not a selection step	both parts
Bounded search (Proposition 7.4)	general theorem	a two-component rendezvous threshold	all parts
The 36 catalog entries (Sections 6 and 7)	certificate specifications	open obligation: the record's fields, for each system; the instances are records of the reference world (Section 8); two entries are refuted proposals and two are deferred questions	the validity predicates and their consequences, and the finite assays by evaluation

Standing hypotheses. Throughout, all records are well typed and carry source locations; no undeclared operation, source or observer resource is used; and results from earlier papers enter only through recorded bridges, which name the source and target and the hypotheses kept, added and lost. These three hypotheses are stated once here and apply to every certificate statement below.

Finite evidence. Many entries are accompanied by an exhaustive enumeration over a declared finite carrier. Such an enumeration establishes the status of every case inside that carrier and nothing about cases outside it. Where a statement is universal, its warrant is the proof.

3.3 Notation

We write $\mathcal{T}_1 \bowtie_{\mathcal{R}} \mathcal{T}_2$ for a join of theory packages $\mathcal{T}_1, \mathcal{T}_2$ under a regime $\mathcal{R}$. The symbol $\bowtie$ distinguishes this object from Foundations V's repair join $\vee$, which is a total lattice operation on quotients of one carrier; a join of theories relates two packages, need not exist, and when it exists must be paid for. In Section 4 a join is modelled by a carrier J with maps $p_A : J \rightarrow A$ and $p_B : J \rightarrow B$; we write $\langle p_A, p_B \rangle : J \rightarrow A \times B$ for their pairing. For a relation $\rightarrow$ on states, $\rightarrow^*$ is its reflexive and transitive closure.

4 Strict joins as non-factorization

This section proves Theorems A and C. The certificate language of Section 6 records a join as strict through flags asserting that a distinction factors on its image neither through either parent, nor through their product, nor through a declared common refinement. Here the first two conditions are given a semantic model and characterized exactly; the third is the subject of Remark 4.9.

4.1 Packages, joins and factorization

Definition 4.1 (Semantic package and join). A *semantic package* with values in X is a set C with an observable map $C \rightarrow X$. A *semantic join* of parents A and B is a set J with maps $p_A : J \rightarrow A$ and $p_B : J \rightarrow B$.

Definition 4.2 (Factorization and split pairs). Let $q : J \rightarrow Y$ and $f : J \rightarrow X$. We say f *factors (globally) through q* if $f = g \circ q$ for some $g : Y \rightarrow X$, and f *factors through q on its image* if $f(j) = g(q(j))$ for all j and some g defined on the reached image $q(J) = \{q(j) : j \in J\}$. A *split pair* for q and f is a pair $j, k \in J$ with $q(j) = q(k)$ and $f(j) \neq f(k)$.

Factorization implies factorization on the image, by restricting g . The image version is the right one here: it requires no value of g at points of Y that q never reaches, and so needs no default element of X .

Theorem 4.3 (Split-pair criterion). *For all maps $q : J \rightarrow Y$ and $f : J \rightarrow X$, f factors through q on its image if and only if no split pair for q and f exists. If J is covered by a finite list and equality on Y and on X is decidable, then a scan of all pairs from the list decides whether a split pair exists.*

Proof. If $f = g \circ q$ on the image and $q(j) = q(k)$, then $f(j) = g(q(j)) = g(q(k)) = f(k)$, so there is no split pair. Conversely, if there is no split pair then f is constant on each fibre of q . For y in the image choose j with $q(j) = y$ and set $g(y) = f(j)$; the value does not depend on the choice, and $f(j) = g(q(j))$ for every j . For the finite statement, a split pair is a pair of points of J , and every point occurs in the list, so the scan finds a split pair exactly when one exists. $\square$

4.2 Strict observables

Definition 4.4 (Strict observable). An observable $f : J \rightarrow X$ of a semantic join is *strict* if it factors on its image through none of p_A , p_B and $\langle p_A, p_B \rangle$.

Theorem 4.5 (Strictness by split pairs). *An observable f of a semantic join is strict if and only if there are split pairs for p_A and f , for p_B and f , and for $\langle p_A, p_B \rangle$ and f .*

Proof. Apply Theorem 4.3 to each of the three maps. $\square$

A split pair for the pairing is also a split pair for each parent, since $\langle p_A, p_B \rangle(j) = \langle p_A, p_B \rangle(k)$ implies $p_A(j) = p_A(k)$ and $p_B(j) = p_B(k)$. So an observable is strict exactly when a single pair of states agrees on both parents and disagrees on the observable. We keep the three clauses separate because the certificate language records them separately.

Example 4.6 (The three-bit join). Let $J = \{0, 1\}^3$ with $p_A(a, b, h) = a$ and $p_B(a, b, h) = b$. The observable $f(a, b, h) = h$ is strict: the states $(0, 0, 0)$ and $(0, 0, 1)$ form a split pair against all three maps. The observable $f(a, b, h) = a$ is not strict: it factors through p_A , hence on its image, so there is no split pair for p_A and f .

Figure 1 shows the eight states grouped by the pairing.

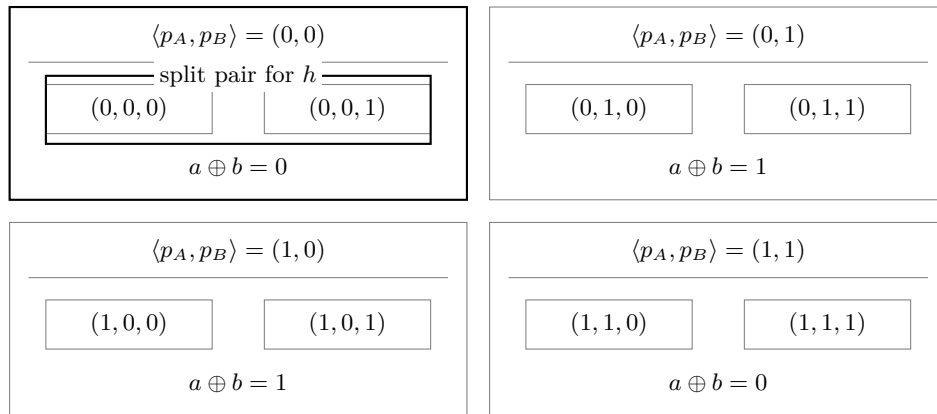


Figure 1. The four fibres of $\langle p_A, p_B \rangle$ in $\{0, 1\}^3$. Each contains both values of h and one value of $a \oplus b$; the outlined states in the $(0, 0)$ fibre form a split pair for h .

The observable $a \oplus b$ of the introduction lies between the two. It has split pairs against each parent but none against the pairing, since (a, b) determines $a \oplus b$; it is therefore not strict. This is the elementary reason the test includes the pairing (Section 1.1).

4.3 Refinements and cheap constructions

Definition 4.7 (Common refinement). A map $q : J \rightarrow R$ is a *common refinement* of the parents if there is $h : R \rightarrow A \times B$ with $h \circ q = \langle p_A, p_B \rangle$.

Proposition 4.8 (Global factorization through the pair passes to every refinement). *If f factors globally through $\langle p_A, p_B \rangle$ (Definition 4.2) and q is a common refinement, then f factors globally through q , and hence on its image.*

Proof. If $f = g \circ \langle p_A, p_B \rangle$ and $h \circ q = \langle p_A, p_B \rangle$, then $f = (g \circ h) \circ q$. □

Remark 4.9 (The limit of strictness). The converse of Proposition 4.8 fails. In Example 4.6 the identity map of J is a common refinement, and the strict observable h factors through it on its image. Strictness is tested against the parents and their pairing only, and says nothing about an arbitrary finer description of the join. A certificate that claims non-factorization through a common refinement must therefore name that refinement and prove the claim for it, and Theorem 4.12 takes that proof as a hypothesis.

The certificate language also refuses three cheap ways of manufacturing a joint distinction: relabelling, rescheduling and coarsening. Each has a semantic counterpart.

Definition 4.10 (Relabelling, scheduling and coarsening). An observable f of a semantic join is

1. *relabelling-only* if it factors on its image through $\sigma \circ \langle p_A, p_B \rangle$ for some bijection σ of $A \times B$;
2. *scheduling-only* for a declared schedule map $s : J \rightarrow S$ if it factors on its image through s ;
3. *coarsening-only* if it factors on its image through $c \circ p_A$ for some map c out of A , or through $c \circ p_B$ for some map c out of B .

Proposition 4.11 (Strictness excludes relabelling and coarsening). *A strict observable is neither relabelling-only nor coarsening-only.*

Proof. A split pair for q and f is a split pair for $c \circ q$ and f , for any map c . The split pairs of Theorem 4.5 for the pairing and for each parent therefore survive composition with σ and with c , and Theorem 4.3 rules out each factorization on the image. □

Scheduling is different: whether f factors on its image through a schedule depends on the schedule, and every observable factors on its image through the identity map of J . A schedule, like a refinement, must therefore be declared.

4.4 The bridge to the certificate language

The certificate language records strictness as an *anti-product witness* (Definition 6.2): a record with a flag for a joint distinction, four flags for factorization on the image through the left parent, the right parent, the declared product and the declared common refinement, three flags for relabelling, scheduling and coarsening, a description and an audit. The witness is valid when the first flag is set, the other seven are clear, the description is non-empty and the audit has an entry. As a record the witness is only an assertion. The following theorem is a soundness check: it shows that the witness's validity predicate is satisfied by a semantic model, with every flag earned by a proof.

Theorem 4.12 (Soundness of the strictness certificate). *Let f be an observable of a semantic join, let $q : J \rightarrow R$ be a common refinement of the parents, and let $s : J \rightarrow S$ be a declared schedule. Suppose f is strict, does not factor on its image through q , and is not scheduling-only for s . Set each flag of an anti-product witness to the truth value of its semantic counterpart: a split pair for $\langle p_A, p_B \rangle$ and f ; factorization on the image through $p_A, p_B, \langle p_A, p_B \rangle$ and q ; and the three conditions of Definition 4.10. With a non-empty description and an audit entry, the resulting witness is valid. In particular, Example 4.6 with $q = \langle p_A, p_B \rangle$ and $s = p_A$ yields a valid witness.*

Proof. Each flag is checked against its semantic counterpart in turn. The joint-distinction flag is set by Theorem 4.5. The flags for the parents and the pairing are clear because f is strict, the refinement flag is clear by hypothesis, the relabelling and coarsening flags are clear by Proposition 4.11, and the scheduling flag is clear by hypothesis. For the example, strictness is Example 4.6; the pairing is a common refinement through the identity of $A \times B$, non-factorization through it is part of strictness, and non-factorization through $s = p_A$ is the first clause of strictness. $\square$

The theorem is close to a restatement. Once strictness and the two extra hypotheses are granted, each flag is settled in one line, by Theorem 4.5, Proposition 4.11 or a hypothesis, and no new construction is involved. Its role is to check the certificate language against the semantics: the Boolean witness asks for nothing that a semantic strict join cannot supply. The two extra hypotheses are genuine, because strictness does not supply them (Remark 4.9).

The schedule $s = p_A$ in the example models a join assembled by running the left parent: the schedule reads a state only through its A -part. It is chosen because it is the simplest declared schedule for which the scheduling hypothesis needs no proof beyond strictness, since non-factorization through p_A is the first clause of strictness. A schedule that sees more of J than the parents do can make the hypothesis fail, as the identity map of J always does, and for such a schedule it must be proved.

The flags are truth values of propositions that need not be decidable, so the witness is defined classically; for finite carriers Theorem 4.3 decides them. The theorem says nothing about the other parts of a strict-join certificate, which record composite objecthood (that the joint candidate is a fixed point of a declared closure, an object in the sense of Foundations I), retention of the parents, source independence and a settled budget (Definition 6.2). Those remain recorded assertions.

5 Admission orders and confluence

This section proves Theorem B. A theory is often extended by admitting items one at a time: definitions, axioms, components, each available once none of its prerequisites is still pending. Whether the result depends on the order of admission is a confluence question, and the classical tools answer it.

5.1 Rewriting and Newman's lemma

Definition 5.1 (Rewriting notions). Let $\rightarrow$ be a relation on a set S and $\rightarrow^*$ its reflexive and transitive closure. Two states y, z are *joinable* if $y \rightarrow^* w$ and $z \rightarrow^* w$ for some w . The relation is *locally confluent* if $x \rightarrow y$ and $x \rightarrow z$ imply that y, z are joinable, and *confluent* if $x \rightarrow^* y$ and $x \rightarrow^* z$ imply that y, z are joinable. It *terminates* if there is no infinite chain $x_0 \rightarrow x_1 \rightarrow \dots$, that is, if the converse relation is well founded. A state x is *normal* if $x \rightarrow y$ for no y .

Theorem 5.2 (Newman's lemma). *A terminating, locally confluent relation is confluent.*

Proof. By well-founded induction on x along the converse of $\rightarrow$, we show that any two paths $x \rightarrow^* y$ and $x \rightarrow^* z$ end in joinable states. If either path is empty the claim is immediate. Otherwise the paths begin $x \rightarrow a \rightarrow^* y$ and $x \rightarrow b \rightarrow^* z$. By local confluence $a \rightarrow^* w$ and $b \rightarrow^* w$ for some w . The induction hypothesis at a joins y and w at some v , and at b joins z and w at some t . Applied once more at a to the paths $a \rightarrow^* w \rightarrow^* v$ and $a \rightarrow^* w \rightarrow^* t$, it joins v and t at some u , and u joins y and z . $\square$

Corollary 5.3 (Unique normal forms). *If $\rightarrow$ is confluent, any two normal states reachable from a common state are equal. If $\rightarrow$ also terminates, every state reaches exactly one normal state.*

Proof. Two normal states reachable from one state are joinable, and a path out of a normal state is empty, so they are equal. Under termination every state reaches some normal state, by well-founded induction: either it is normal, or it steps to a state that reaches one. $\square$

For a finite system, local confluence can be decided mechanically.

Proposition 5.4 (Finite decision of local confluence). *Let S be covered by a finite list, with decidable equality and a decidable one-step relation. Then reachability $x \rightarrow^* y$ holds exactly when y lies in every subset of the list that contains x and is closed under steps, and this can be checked by enumerating the subsets. Joinability is decided by searching the list for a common reachable state, and local confluence by checking every one-step peak.*

Proof. The set of states reachable from x is itself closed under steps and contains x , so a state in every such subset is reachable. Conversely, a closed set containing x contains everything reachable from x , by induction on the path. The rest is a finite search over the covering list. $\square$

Remark 5.5 (Efficient search). The subset enumeration is exponential in the length of the list. It is the form the Lean proof checks, because the characterization by closed subsets makes its correctness easy to verify. In practice reachability is decided by graph search: a breadth-first or depth-first search from x visits each state at most once and computes the set of states reachable from x with at most n^2 tests of the step relation, for a list of length n . Local confluence is then checked by one such search from each end of each one-step peak.

Example 5.6 (Termination is needed). On four states a, b, c, d let the only steps be $b \rightarrow a$, $b \rightarrow c$, $c \rightarrow b$ and $c \rightarrow d$. Every one-step peak is joinable: the peak $b \rightarrow a$, $b \rightarrow c$ joins at a via $c \rightarrow b \rightarrow a$, and the peak $c \rightarrow b$, $c \rightarrow d$ joins at d via $b \rightarrow c \rightarrow d$. The relation is not confluent, since $b \rightarrow a$ and $b \rightarrow c \rightarrow d$ reach the distinct normal states a and d . The relation does not terminate: $b \rightarrow c \rightarrow b \rightarrow \dots$.

5.2 Admission systems

Definition 5.7 (Admission system). An *admission system* over a set I of items consists of a prerequisite relation $p \prec i$ on I and an audit map from finite lists of items to a set of labels. A state is the list of items still pending. An item i is *enabled* in a state s if i is pending and no prerequisite of i is pending, and the admission step removes an enabled item: $s \rightarrow s \ominus i$, where $s \ominus i$ deletes the first occurrence of i from the list s .

The model tests only that no prerequisite is still pending. A prerequisite that was never in the initial list therefore counts as satisfied. When every prerequisite of every pending item lies in the initial list, “no prerequisite of i is pending” means that every prerequisite of i has been admitted. Removing an item never makes an item pending again. This monotonicity, that admission can only enable, is what the next theorem uses.

Theorem 5.8 (Admission order does not matter). *In every admission system:*

1. every admission run terminates;
2. if $s \rightarrow y$ and $s \rightarrow z$, then $y = z$ or there is w with $y \rightarrow w$ and $z \rightarrow w$;
3. the admission relation is confluent;
4. from any state s , every maximal admission order reaches the same terminal state, and so the same audit value.

If the initial list has no repetitions, neither does any state reached from it.

Proof. Termination and one-step commutation give confluence by Newman's lemma, and uniqueness of the terminal state follows. (1) Each step shortens the pending list. (2) Let $y = s \ominus i$ and $z = s \ominus j$ with $i \neq j$. Removing j keeps i pending and pending no new item, so i is still enabled in z ; likewise j is enabled in y . Both orders then reach $(s \ominus i) \ominus j = (s \ominus j) \ominus i$. (3) By (2) the relation is locally confluent, and by (1) and Theorem 5.2 it is confluent. (4) This is Corollary 5.3; equal terminal states have equal audit values. The last statement holds because removing an element preserves the absence of repetitions. $\square$

The terminal state need not be empty. Items on a cycle of prerequisites, or depending on such items, are never enabled and remain pending. The theorem says they remain pending in every order.

Example 5.9 (One disabling guard). Take items a and b and a guarded step in which a may always be admitted but b may be admitted only while a is still pending. From $\{a, b\}$ the relation terminates. Admitting a first reaches $\{b\}$, which is terminal because b is now blocked. Admitting b first reaches $\{a\}$ and then $\emptyset$. The two terminal states differ, and so do their lengths. The relation is not locally confluent, since the peak $\{a, b\} \rightarrow \{b\}$, $\{a, b\} \rightarrow \{a\}$ cannot be joined.

Figure 2 shows this example beside that of Example 5.6.

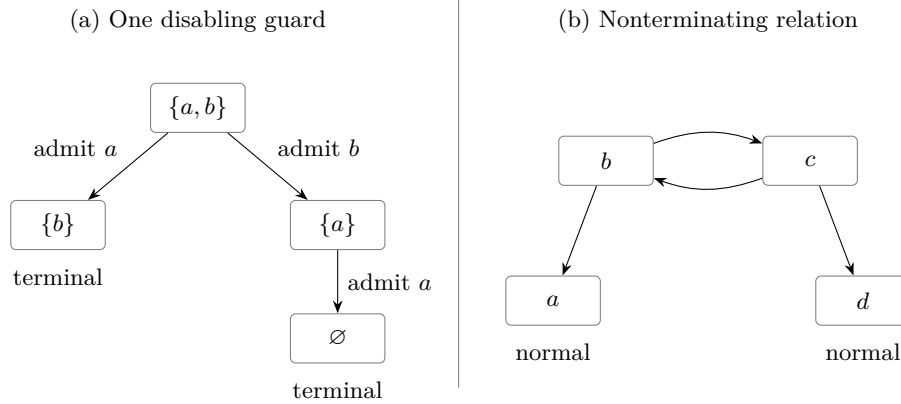


Figure 2. (a) Admitting a from $\{a, b\}$ leaves terminal $\{b\}$, while admitting b first leads to terminal $\emptyset$. (b) The steps $b \rightarrow a$, $b \rightarrow c$, $c \rightarrow b$, and $c \rightarrow d$, with normal states a and d .

The guard violates the monotonicity of admission: admitting a disables b .

Example 5.10 (Admission without constraints). With two items and no prerequisites, both orders admit both items and reach the empty terminal state.

The catalog records a finite admission system as a list of critical-pair records, each marked legal or not, joinable or not and audit-equivalent or not, and states that a critical-pair criterion is equivalent to confluence at the declared scope. Both sides of that equivalence are predicates

on the same list of records, so the statement is a reformulation rather than a confluence theorem (Section 7.3). Theorem 5.8 is the corresponding semantic result, and Example 5.9 proves that different orders from one initial state can reach different terminal states once an admission may disable another.

6 The certificate language

A claim about interaction is admitted into the catalog only as a certificate: a record with one field for each thing the claim asserts, usually with an audit. The catalog has twenty-three kinds of certificate, in four clusters. In every cluster validity has the same shape: the fields the claim names are present, the Boolean flags it needs are set, the stated implications between flags hold, and, where the record has one, the audit has an entry. The supplement gives the validity predicate and the Lean fields of every certificate (Supplement Section S5).

Access and admission. An access record gives a content seven Boolean coordinates (Definition 6.1). An admission step is a typed transition of one of five kinds; an admission regime records the routes by which a first extension may be authorized. Neutral provisioning and prospective commitment records certify a source from outside the system and a registration made before its evidence. A bridge contract records what a transport between theories keeps, adds and loses; transport is licensed only by an accepted, well-formed bridge, never by a citation.

Contact and join. A contact surface and a contact witness record that two theories touched (Definition S3.1). A join entry records a candidate joint theory with its parents, ledgers and one of seven statuses (Definition S3.2). The strict-join certificate records the claim that a join is more than its parents (Definition 6.2). Three further records carry provenance and price. The source-independence gate compares the ancestries of the two sides: each source traces back to a root source, and the sides have *disjoint roots* when their roots differ. The cost ledger records what the join paid. An observer-occupancy record states how much of an observer’s or instrument’s bounded capacity the interaction uses, its *occupancy*, and how much of that is charged to the ledger.

Enablement and descent. An enablement record states that one theory makes an operation available to another (Definition S3.3). A transmission record carries a payload between layers. An operational profile and a negative-evidence record fix what a rule’s reachability and a failed search establish, and a coverage certificate is what a claim of non-interaction must carry.

Dynamics and residue. A route comparison records two orders of the same members (Definition S3.4), a drive certificate is what an arrow claim needs (Definition S3.5), a residual ledger records what an interaction leaves undischarged, and a contact-quantity record states what a claimed conservation includes.

The simplest record shows what a certificate looks like.

Definition 6.1 (Access record). An *access record* for a content assigns seven Boolean coordinates: whether it is expressible, present, exposed, recoverable, admissible, reachable and occurrent. The record is *coherent* when

occurrent $\Rightarrow$ reachable $\Rightarrow$ admissible $\Rightarrow$ present $\Rightarrow$ expressible, recoverable $\Rightarrow$ exposed $\Rightarrow$ present,
and its audit is non-empty.

The implications run one way only, and the reference world contains a coherent record for each failed converse (Section 7). Of the $2^7 = 128$ assignments of the seven coordinates, exactly 14 satisfy these two chains of implications, the *implication spine* of the record. A single record already distinguishes readings that ordinary usage runs together. Consider a content that is **expressible**, **present** and **admissible** but not **exposed**, **recoverable**, **reachable** or **occurrent**. The record is coherent. It is not latent in a way that makes it available on demand, because it is not **reachable**. It is not hidden in a way that looking harder would uncover, because exposure is a property of the interface, not of the observer. It is not merely unobserved, because **occurrent** records what happened in the system. And it is not forbidden, because it is **admissible**.

The other records are defined in Supplement Section S3, except the strict-join certificate, which carries most of the weight of the catalog; we give it in full, filled in for the three-bit join of Example 4.6.

6.1 The strict-join certificate

Definition 6.2 (Strict-join certificate). A *strict-join certificate* records a join candidate, composite objecthood, recoverability of each parent, an anti-product witness, source independence, a settled budget, a directionality flag and an audit. The *anti-product witness* has a flag for a joint distinction; flags for factorization on the image through the left parent, the right parent, the declared product and a declared common refinement; flags for relabelling-only, scheduling-only and coarsening-only constructions; a description; and an audit. The witness is valid when the joint-distinction flag is set, the seven other flags are clear, the description is non-empty and the audit has an entry. The certificate is valid when the candidate is well formed, the objecthood, retention, independence and budget flags are set, the witness is valid and the audit has an entry. Directionality is not required: a strict join does not carry an arrow.

As a record, the anti-product witness is an assertion of non-factorization: its flags are entered, not derived. What makes it more than an assertion is Theorem 4.12: a semantic strict join that is also proved not to factor, on its image, through a declared common refinement and a declared schedule produces a valid witness, with each flag equal to the truth value of its semantic predicate. Strictness alone does not supply those two proofs (Remark 4.9). The other fields of the certificate (objecthood, parent retention, source independence and budget) have no semantic model here and remain recorded assertions.

Example 6.3 (The certificate of the three-bit join). For the three-bit join of Example 4.6 with the observable $f(a, b, h) = h$, take the pairing $\langle p_A, p_B \rangle$ as the declared common refinement and p_A as the declared schedule. Theorem 4.12 fills the anti-product witness: the joint-distinction flag is set by the split pair $(0, 0, 0)$, $(0, 0, 1)$, and the seven other flags are clear. A well-formed join candidate, objecthood of the composite, recoverability of a and b , disjoint source roots, a settled budget and a non-empty certificate audit must then be supplied as recorded assertions; once they are, the certificate is valid by Definition 6.2, and the directionality flag may stay clear. Only the anti-product witness is derived; the rest is what a user of the certificate must supply.

7 What the certificates separate and refuse

The catalog has 36 entries. This section states what they establish, sorted by kind: separations exhibited in the reference world, statements that follow from the definitions, refusals of credit, two refuted proposals and two deferred questions. The supplement states each entry (Supplement Section S6).

7.1 Separations

The main use of the language is to keep apart properties that are easily conflated. For 27 such pairs — sound versus reachable, contact versus strict join, enablement versus descent, route residue versus an arrow, and so on — the finite reference world of Section 8 contains a record that has the first property and lacks the second. These separations hold between certificate fields, so they show only that the fields are independent as records; Table S2 lists them.

Four of them have semantic counterparts proved in this paper. Proposition 4.8 explains why an observable that factors globally through the pairing gains nothing from a common refinement, and Remark 4.9 states what strictness leaves open. Proposition 4.11 explains why relabelling produces no strict content. The seed-partition row is a record that stipulates two seed partitions and stores unequal terminal-package fields. It is a different statement from Example 5.9, which proves that two admission orders from the same initial state reach different terminal states because one admission disables another. When admission only enables, Theorem 5.8 gives a unique terminal state from each state. The enablement row also has a semantic counterpart (Proposition 7.1).

7.2 Semantic models

Four of the entries above are also proved about explicit transition systems rather than about records.

Proposition 7.1 (Enablement without descent). *Let the lower theory have states $\{0, 1, 2\}$ and a step $a \rightarrow a'$ whenever $a < a'$. Form an upper state from each lower state a as the vector of states reachable from a , and let the upper theory have a step between two formed states exactly when a lower step joins their seeds. Then*

1. *the three formed states are distinct, and an upper step exists exactly when a lower step supplies it; in particular the formed state of 0 steps to the formed state of 1;*
2. *the observable “0 is reachable” does not factor on its image through the quotient “1 is reachable”: the formed states of 0 and 1 agree on the quotient and differ on the observable.*

An observable that is constant on the fibres of a quotient does factor through it.

Proof. Formation is injective because the vector of a has first true entry at a ; an upper step between formed states is by definition a lower step between their seeds. The formed states of 0 and 1 are $(1, 1, 1)$ and $(0, 1, 1)$, a split pair for the quotient and the observable, so Theorem 4.3 applies. The last sentence is the converse direction of the same theorem. $\square$

Figure 3 shows the two layers and the split pair.

The model is small, but it separates the two properties as statements about transitions, not as independent record fields: the upper step is enabled by the lower dynamics, and what the upper theory observes is still not a function of the lower quotient.

Proposition 7.2 (No free join). *Consider events that require and produce capabilities, and call a finite sequence of events lawful when every capability an event requires is initially available or produced by an earlier event. If a join j requires a capability c that is not initially available, and j is the only event that produces c , then j occurs in no lawful sequence. If every capability that j requires is initially available, the sequence consisting of j alone is lawful.*

Proof. Induction on the lawful sequence. When j is appended, c must be initially available, which it is not, or produced by an earlier event, which would be an earlier occurrence of j ; the induction hypothesis excludes that. The second statement is immediate from the definition of a lawful sequence;

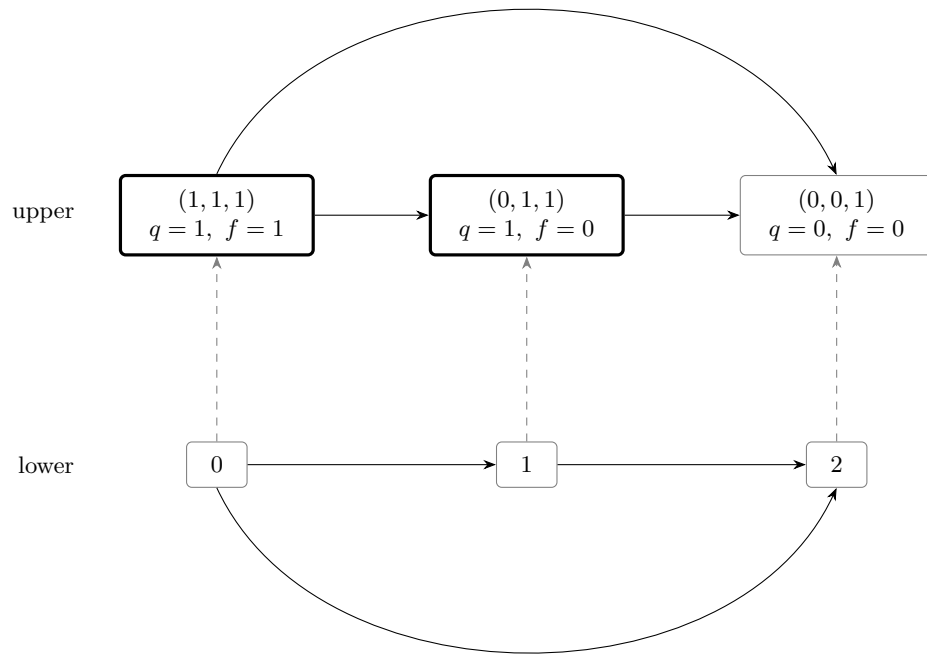


Figure 3. Lower steps and their induced upper steps between formed reachability vectors. Dashed arrows map each lower state to its vector; the outlined vectors agree on q (“1 reachable”) and differ on f (“0 reachable”).

the formal development checks it for a one-event system in which the join's capability is initially supplied. $\square$

Proposition 7.3 (Selection creates no fact). *Let a state be a set of lower facts, and let a selection step replace a state S by $S \cap K$ for some set K . After any finite sequence of selection steps, every fact present was present at the start. A step that inserts a fact into the empty state is not a selection step.*

Proof. $S \cap K \subseteq S$, and inclusion is transitive along the sequence. $\square$

The proposition is immediate once selection is modelled as intersection; its use is to fix what a pure downward selection is allowed to do.

Proposition 7.4 (Bounded search). *Let a finite list of states be scanned for a decidable target, and let the reachable set be the states reachable from a start within h steps.*

1. *If every listed state is reachable within h steps and the scan finds the target, some state reachable within h steps satisfies it.*
2. *If every state reachable within h steps is listed and the scan finds nothing, no state reachable within h steps satisfies the target; if moreover every reachable state is reachable within h steps, no reachable state does.*

Proof. Both parts compare what the scan inspects with what is reachable: the first uses that every listed state is reachable, the second that every reachable state is listed. (1) The scan returns a listed state satisfying the target. (2) A reachable target state would be listed, so the scan would find it. $\square$

A negative search result is thus exactly as strong as the coverage of its list and the closure of its horizon. For two Boolean components that each step from false to true, one step at a time, the joint state (true, true) is not reachable within one step and is reachable within two; this is a rendezvous threshold, not yet a model of a channel carrying contact between two theories.

7.3 Statements that follow from the definitions

The following catalog entries are definitions, or statements that follow in a line from the definition of the certificate they concern. The first is stated as a proposition because the later refusals use it.

Proposition 7.5 (Bootstrap). *In a closed admission regime with no admitted seed and no reachable generator, no first extension is authorized. Each of the three routes to authorization (an admitted seed, a reachable generator, open external provision) authorizes a first extension in some regime.*

Proof. Authorization is defined as the disjunction of the three routes, and open external provision requires the family to be open. The escapes are exhibited by three regimes, one for each route. $\square$

The content of the proposition is the list of routes, which fixes exactly what a first extension in a closed regime needs. The other entries of this kind — neutral seeds, endogenous enablement, downward selection, horizons, certified non-interaction, composition of enablement, confluence at a declared scope, join status, order residue, the join-created residual and the cross-family corollaries — are stated in Section S4.1; the four with semantic counterparts are Propositions 7.1 to 7.4.

7.4 Refusals

Eleven entries are refusals: a credit is withheld when a field its definition requires is absent, and named routes restore it. Table 3 lists them. In each case the eligibility condition for the credit contains the missing field, so the refusal is immediate from the definition. The content is the list of escape routes, each of which is exhibited by a configuration that earns the credit. One refusal, occurrence from reachability, is also accompanied by an explicit counterprofile.

Table 3. Refusals of credit and their escape routes.

Refused credit	Missing field	Escape routes
first extension in a closed regime	seed or reachable generator	admitted seed; reachable generator; open external provision
independence-sensitive join from resemblance	witnessed independent contact	witnessed independent contact
retrospective self-certification	prospective certificate	prospective certificate
total-lens transfer to a partial target	certified adapter	certified adapter
native credit with an unpriced observer	charged occupancy	external observer; zero occupancy
join from contact alone	composite, retention and anti-product witness	complete strict-join certificate
strictness from a product or common refinement	anti-product witness	anti-product witness
source independence from one lineage	disjoint roots	disjoint-root certificate
positive-cost join without payment	payment	paid channel (a certified zero-cost channel credits only a join of amount zero)
arrow from route residue	drive certificate	independently driven arrow
occurrence from reachability	firing	a fired occurrence

Three further refusals are worth recording with them. A join cannot create the first capability needed to execute itself, cannot manufacture strict content by relabelling, and cannot receive positive-cost formation for nothing. The second follows from Proposition 4.11 and the third is the payment refusal above. The first is Proposition 7.2: a capability that only the join produces cannot authorize the join’s own execution.

An observer or instrument that uses bounded shared capacity cannot certify native or endogenous formation while its occupancy is under-charged or its source role or cost is off the ledger. Observation is not refused; unrecorded observation is.

7.5 Two refuted proposals

Remark 7.6 (Refinement monotonicity is false). The unconditional claim that refining a parent preserves the join is refuted at the level of records. A refinement record carries an effect label (preserve, strengthen, weaken or destroy) and preservation flags. A well-formed record labelled *destroys* exists, so the certificate language does not permit the unconditional claim. What survives is a classification of refinement records by their label, and all four labels have well-formed record witnesses. As a contrast in observable factorization, not a model of refinement acting on a join, the three-bit join shows both behaviours: the pairing, a coarse common refinement, loses the observable

h , while the identity refinement retains it (Remark 4.9). No model of refinement acting on a join is given.

Remark 7.7 (Two contact quantities disagree). One might expect contact to carry a conserved scalar degree. A contact quantity records five coordinates: access, retention, novelty, observer occupancy and residual. For the vector $(1, 1, 1, 0, 1)$ the plain sum is 4, while the sum that weights novelty by 2 and residual by 3 is 7, so the two plausible degrees disagree. An apparent conservation that omits the observer’s occupancy fails the accounting certificate, while an exact scoped conservation certificate, with every coordinate included, is available. No universal statement about conserved quantities is proved in either direction: the two scalars disagree, and the accounting stays local.

7.6 Two deferred questions

Remark 7.8 (Categorical reduction). Whether a join of theories reduces to a product, pullback or pushout [13] is not settled here. The catalog’s categorical certificate is a checklist of flags (identities checked, composition closed, associativity checked, a universal property recorded) with no morphisms or composition, so it supports no categorical statement. What is proved is the set-level fact of Theorem 4.5: a strict observable does not factor, on its image, through the pairing $J \rightarrow A \times B$, which is the map into the product of sets. A categorical treatment would need hom-sets, composition and a universal property quantified over mediating maps.

Remark 7.9 (Primitive-operation algebra). A generators-and-relations presentation of the primitive operations is not attempted. The part the paper uses is the monoid of resource deltas described above.

8 The reference world and finite evidence

The catalog is anchored by one finite model, the *Two-Theory World*: two theory packages with access records, admission transitions, contact surfaces, join entries, enablement chains, budget ledgers and audits, small enough to enumerate and rich enough that every certificate has an instance in it. Named scenarios exercise each certificate, and named countermodels each refute one tempting inference; the record-level separations of Table S2 are among them. A detector contract, with signal, null and falsifier cases and controls for same source, no contact, rescheduling and relabelling, fixes how each certificate is tested, and finite assays enumerate declared carriers exhaustively. All of this shows how the certificates read, not that they are the right ones or that any law holds; the scenarios, countermodels, contract and assay counts are in Supplement Section S7.

9 Scope and limitations

Table 4 lists the inferences a reader is most likely to draw from this paper and what blocks each. The full register of recorded non-implications is in Supplement Section S6.

Table 4. Tempting inferences and what blocks them.

Inference	What blocks it
The catalog’s 36 entries are 36 theorems.	Most are definitions, consequences of definitions or refusals (Sections 3.2 and 7.3); the theorems are those of Sections 4 and 5.

Inference	What blocks it
A valid strict-join certificate shows that a strict join exists.	The certificate's flags are recorded assertions. Theorem 4.12 shows they can be earned by a semantic strict join proved not to factor through a declared refinement and schedule; objecthood, parent retention, independence and budget remain assertions.
A strict observable is new relative to every description of the join.	Strictness is tested against the parents and their pairing only (Remark 4.9).
An observable that neither parent determines is strict.	It may factor on its image through the pairing, as $a \oplus b$ does (Section 4).
Admission order never matters.	It does not matter when admission can only enable, which is sufficient but not shown necessary (Theorem 5.8); one disabling guard gives two terminal states (Example 5.9).
Local confluence gives confluence.	Not without further hypotheses: with termination it does (Theorem 5.2), and without it it can fail (Example 5.6).
Every item is eventually admitted.	Items on a cycle of prerequisites are never enabled (Theorem 5.8).
The catalog's critical-pair criterion proves confluence.	It restates a predicate on critical-pair records (Section 7.3).
Joining theories reduces obstructions.	The residual ledger records residuals created at the composite interface; the record is an attribution, not a derived fact (Section 7.3).
Contact, a shared origin or a shared carrier is interaction.	Each is separated from shared access, transport and join (Table S2).
Enablement explains what it enables.	Enablement is separated from descent and from sufficiency (Table S2).
Order dependence, or a strict join, gives an arrow of time.	An arrow requires a drive certificate (Definition S3.5).
A join is a categorical product.	No categorical statement is made; only the set-level pairing appears (Remark 7.8).
No quantity is conserved under contact.	Two particular scalars disagree; nothing universal is proved (Remark 7.7).
A negative search result is universal.	Its force is that of its search family, horizon and detector (Section 7.3).
The finite assays prove the laws.	They are exhaustive over their declared carriers only (Section 8).
The results apply to particular real theories.	No instantiation is carried out; each would need its own regime, carriers, witnesses and audit.

10 Formalization

The results are accompanied by a development in Lean 4.28.0 that uses no external library. It has two parts.

The certificate library. The certificate language and its reference world comprise 1,477 public declarations, of which 732 are theorems. An axiom report was recorded for every one of these theorems. None depends on the axiom of choice or on an unfinished proof; the others use at most propositional extensionality and quotient soundness, and the counts are in Supplement Section S8. Most of these theorems concern records and are short consequences of their definitions, as Section 7.3

describes; the kernel confirms that each follows, not that it is deep. One record-level construction, a commuting square used to record faithful transmission between layers, is stated for types in Lean’s lowest universe only.

The semantic modules. Sections 4 and 5 are formalized in three further modules, and the propositions of Section 7.2 in five of six further modules; their axiom reports are listed in Supplement Section S8. Newman’s lemma and the uniqueness of normal forms depend on no axioms. The axiom of choice enters in four places: in choosing a representative of each fibre in Theorem 4.3; in choosing a successor when a terminating relation reaches a normal state; in the proof of the finite reachability check (Proposition 5.4), which uses the set of reachable states classically although the check itself is computable; and through the standard library’s lemmas on removing an element from a list. The flags in Theorem 4.12 are truth values of propositions and are defined classically. These modules are universe-polymorphic; they contain no unfinished proof, no added axiom and no proof by compiled evaluation. The library builds with them included.

What is not formalized. Two statements are argued on paper only: that $a \oplus b$ factors on its image through the pairing but through neither parent (Section 4), and the relation of this work to the external literature (Section 2). The formalization also cannot establish that the definitions capture what they are meant to capture. The kernel checks that each encoded statement follows from its encoded hypotheses [8, 19]; whether a certificate is the right record of an interaction is argued in the paper.

The supplement gives the Lean counterpart of every numbered item (Supplement Section S2), the axiom reports (Supplement Section S8) and the commands that rebuild the library, the finite evidence and the reports (Supplement Section S9).

References

- [1] Martín Abadi and Leslie Lamport. Composing Specifications. *ACM Transactions on Programming Languages and Systems*, 15(1):73–132, 1993. doi:10.1145/151646.151649.
- [2] Bowen Alpern and Fred B. Schneider. Defining Liveness. *Information Processing Letters*, 21(4):181–185, 1985. doi:10.1016/0020-0190(85)90056-0.
- [3] Thomas William Barrett and Hans Halvorson. Morita Equivalence. *The Review of Symbolic Logic*, 9(3):556–582, 2016. doi:10.1017/S1755020316000186.
- [4] Albert Benveniste, Benoît Caillaud, Dejan Nickovic, Roberto Passerone, Jean-Baptiste Raclet, Philipp Reinkemeier, Alberto L. Sangiovanni-Vincentelli, Werner Damm, Thomas A. Henzinger, and Kim G. Larsen. Contracts for System Design. *Foundations and Trends in Electronic Design Automation*, 12(2–3):124–400, 2018. doi:10.1561/10000000053.
- [5] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 1579 of *Lecture Notes in Computer Science*, pages 193–207. Springer, 1999. doi:10.1007/3-540-49059-0_14.
- [6] Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem, editors. *Handbook of Model Checking*. Springer International Publishing, Cham, 2018. ISBN 978-3-319-10574-1. doi:10.1007/978-3-319-10575-8.
- [7] Luca de Alfaro and Thomas A. Henzinger. Interface Automata. In *Proceedings of the 8th European Software Engineering Conference Held Jointly with the 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 109–120. ACM, 2001. doi:10.1145/503209.503226.
- [8] Leonardo de Moura and Sebastian Ullrich. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction—CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi:10.1007/978-3-030-79876-5_37.
- [9] Joseph A. Goguen and Rod M. Burstall. Institutions: Abstract Model Theory for Specification and Programming. *Journal of the ACM*, 39(1):95–146, 1992. doi:10.1145/147508.147524.
- [10] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance Semirings. In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, pages 31–40. ACM, 2007. doi:10.1145/1265530.1265535.
- [11] Gérard Huet. Confluent reductions: Abstract properties and applications to term rewriting systems. *Journal of the ACM*, 27(4):797–821, 1980. doi:10.1145/322217.322230.
- [12] Donald E. Knuth and Peter B. Bendix. Simple word problems in universal algebras. In John Leech, editor, *Computational Problems in Abstract Algebra*, pages 263–297. Pergamon Press, Oxford, 1970. doi:10.1016/B978-0-08-012975-4.50028-X.
- [13] Saunders Mac Lane. *Categories for the Working Mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer, New York, 2 edition, 1998. ISBN 978-0-387-98403-2. doi:10.1007/978-1-4757-4721-8.

- [14] Maël Montévil and Matteo Mossio. Biological Organisation as Closure of Constraints. *Journal of Theoretical Biology*, 372:179–191, 2015. doi:10.1016/j.jtbi.2015.02.029.
- [15] Greg Nelson and Derek C. Oppen. Simplification by Cooperating Decision Procedures. *ACM Transactions on Programming Languages and Systems*, 1(2):245–257, 1979. doi:10.1145/357073.357079.
- [16] M. H. A. Newman. On theories with a combinatorial definition of “equivalence”. *Annals of Mathematics*, 43(2):223–243, 1942. doi:10.2307/1968867.
- [17] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The Preregistration Revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi:10.1073/pnas.1708274114.
- [18] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, 2 edition, 2009. ISBN 978-0-521-89560-6. doi:10.1017/CBO9780511803161.
- [19] Robert Pollack. How to Believe a Machine-Checked Proof. In Giovanni Sambin and Jan M. Smith, editors, *Twenty Five Years of Constructive Type Theory*, pages 205–220. Oxford University Press, 1998. doi:10.1093/oso/9780198501275.003.0013.
- [20] Donald Sannella and Andrzej Tarlecki. *Foundations of Algebraic Specification and Formal Software Development*. Monographs in Theoretical Computer Science. An EATCS Series. Springer, Berlin, Heidelberg, 2012. ISBN 978-3-642-17336-3. doi:10.1007/978-3-642-17336-3.
- [21] Ioannis Tsiokos. To Throw a Stone with Six Birds: On Agents and Agenthood, 2026. Six Birds Theory corpus; source of the distinction between enablement and within-layer causation.
- [22] Ioannis Tsiokos. Six Birds: Foundations of Emergence Calculus, 2026. Six Birds Theory, Foundations I.
- [23] Ioannis Tsiokos. Six Birds Foundations II: Admissibility, Meta-Theory, and the Exact-Six Program, 2026. Six Birds Theory, Foundations II.
- [24] Ioannis Tsiokos. Six Birds Foundations III: A Finite Audited Interaction Calculus for SBT, 2026. Six Birds Theory, Foundations III.
- [25] Ioannis Tsiokos. Six Birds Foundations IV: A Catalog of Layer-Agnostic Structural Laws, 2026. Six Birds Theory, Foundations IV.
- [26] Ioannis Tsiokos. Six Birds Foundations V: Endogenous Closure — A Catalog of Structural Laws for Living, Cognitive, and Social Systems, 2026. Six Birds Theory, Foundations V.
- [27] Ioannis Tsiokos. Six Birds Foundations VI: A Catalog of Dynamical Structural Laws, 2026. Six Birds Theory, Foundations VI.
- [28] Ioannis Tsiokos. Six Birds: No-Go Theorems for Audited Emergence, 2026. Six Birds Theory corpus.
- [29] Ioannis Tsiokos. Strict-Test-to-Event-Package Admissibility on Fixed Support in Six Birds Theory, 2026. Six Birds Theory corpus.

- [30] James Owen Weatherall. Why Not Categorical Equivalence? In Judit Madarász and Gergely Székely, editors, *Hajnal Andréka and István Németi on Unity of Science: From Computing to Relativity Theory Through Algebraic Logic*, volume 19 of *Outstanding Contributions to Logic*, pages 427–451. Springer, 2021. doi:10.1007/978-3-030-64187-0_18.